

Self-Evolving Security Framework for LLM Agents via Verifiable Guardrail Tools

First Author^{1,2*}, Second Author^{2,3†} and Third Author^{1,2†}

^{1*}Department, Organization, Street, City, 100190, State, Country.

²Department, Organization, Street, City, 10587, State, Country.

³Department, Organization, Street, City, 610101, State, Country.

*Corresponding author(s). E-mail(s): iauthor@gmail.com;
Contributing authors: iiauthor@gmail.com; iiiauthor@gmail.com;

[†]These authors contributed equally to this work.

Abstract

With capabilities expanding rapidly, large language model (LLM)-driven agents are being deployed in high-stakes domains such as finance, healthcare, and industry, which bring significant risks. Mainstream defense frameworks rely on static hard-coded rules or predefined tools, limiting their generalization to evolving or unknown scenarios. Inspired by the human capacity for continual self-reflection and knowledge refinement, we propose DEFEND, a self-evolving defense framework. It generates executable Python guardrail tools in response to the runtime behavior of agents and treats these tools as evolvable artifacts that are continuously updated through lifelong interaction. Meanwhile, an independent auditing mechanism ensures the safety of each evolved tool, enabling the framework to maintain trustworthiness and consistency while dynamically evolving. The framework demonstrates excellent performance across various domains, promoting unification in the field of agent defense.

Keywords: self-evolution, agent safety, agentic system

Humans excel at reflecting, summarizing knowledge and experience from their actions, continuously learning and applying these insights to future endeavors. This capacity for "self-evolution" is key to the ever-improving learning abilities of humans and serves as a crucial pathway toward achieving general intelligent agents. In recent years, agents powered by large language models (LLMs) have demonstrated remarkable capabilities

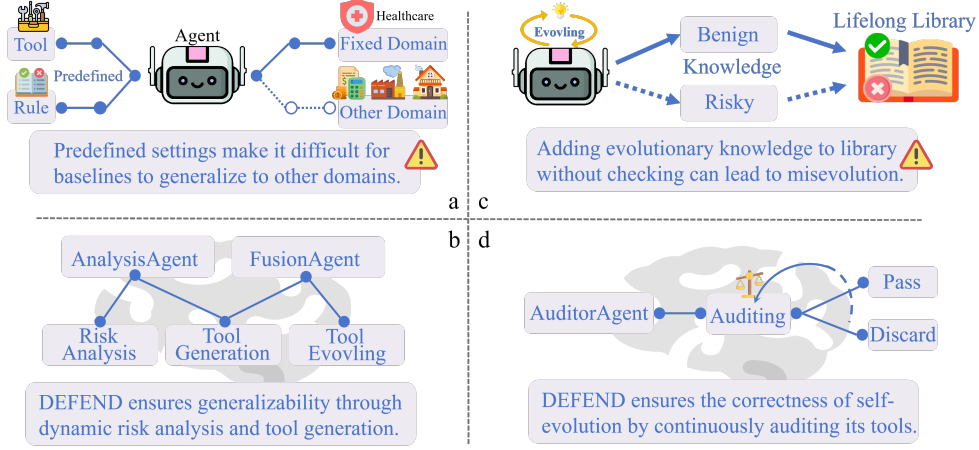


Fig. 1: The limitation of existing methods (a,c) and DEFEND’s solution (b,d).

across diverse tasks, yet they remain essentially static in nature [1]. As more agents are deployed in open, interactive environments, evolving knowledge domains and constantly updated tasks pose challenges to this static paradigm. Self-evolutionary agents are defined as systems capable of dynamically and autonomously improving their own states through self-evolution strategies based on trajectories and feedback generated from interactions with their environment, with the goal of maximizing cumulative task utility [2]. Their emergence represents a paradigm shift from static to dynamic, spanning personalized applications, adaptive reasoning, and internal parameter updates. Self-evolution is becoming a new cornerstone on the path toward super-intelligent artificial intelligence (SAI).

Agents based on LLMs are profoundly transforming human-computer interaction. Capable of autonomously managing files, invoking tools, executing operations, and interacting with their environments [3], they are now widely applied across various aspects of daily life, significantly enhancing convenience in fields such as finance [4], healthcare [5], and home automation [6]. However, as the autonomous capabilities of these agents continue to advance, more severe security challenges have emerged. The first major challenge is that inappropriate operations by an agent can lead to greater dangers. For instance, a medical agent might abuse its permissions to leak patient information [7], or an autonomous driving agent could misinterpret users’ request and perform a hazardous maneuver [8, 9]. The second challenge is the continuous expansion of the defensive perimeter required for agent security. Although some existing methods have shown promising defensive results, they largely depend on hard-coded rules, fixed domains, and pre-defined tools, making them difficult to generalize to other domains with different security requirements [10–12], shown in Figure 1.a. Consequently, there is an urgent need for a unified protection framework that can demonstrate excellent defensive performance across different tasks, domains, and environments while possessing strong generalization capabilities and portability.

To address this issue, we revisit agent safety from the perspective of self-evolutionary agents, shown in Figure 1.b. Traditional defense frameworks are deeply tied to specific domains due to their static nature. Some tentative generalization strategies include injecting domain-specific background knowledge and manually designating trusted contexts [13]. These methods still fundamentally require human intervention, thereby limiting their applicability. Self-evolution offers a new angle to overcome this inherent limitation. Unlike static, predefined strategies, an agent protection framework endowed with self-evolutionary capabilities can actively extract deep security patterns from interaction trajectories through continuous engagement with its environment. In this dynamic process, domain-specific background knowledge and protection strategies are not pre-injected or statically defined but are instead autonomously refined by the agent itself. This self-induction and abstraction, enables the agent to move beyond simple memorization of known threats and develop transferable, generalizable response mechanisms. As a result, it demonstrates robust and adaptive generalization capabilities when encountering novel attacks or operating in previously unseen domains.

For methods based on self-evolution, a significant challenge lies in ensuring that evolution does not go astray [14]. Agents face the risk of mis-evolution, and updates to the knowledge library therefore require validation, shown in Figure 1.c. Some works lack careful auditing of newly acquired knowledge, implicitly assuming that “newer is better”. This strategy, which defaults to treating new knowledge as beneficial and directly incorporating it into updates, is often the root cause of misevolution [14, 15]. Our experiments show that even when safety constraints are incorporated into tool design, the guardrail tools evolved by an agent may still pose risks—a phenomenon we define as *Vulnerabilities in Guardrail Tools (VGT)*. Safety is not a one-time attribute but a dynamic process that requires ongoing verification throughout evolution [3]. Therefore, to build a truly trustworthy self-evolutionary security mechanism, it is crucial to establish an independent auditing system capable of evaluating the new knowledge generated during self-evolution process, shown in Figure 1.d.

Tool calling extends the capability boundaries of agents. Currently, tool calling is primarily used to empower LLMs, enabling them to handle more complex real-world tasks, such as retrieving real-time information through search tools [16], navigating web pages by controlling browsers [17], and operating physical devices [18]. Self-evolutionary agent systems go a step further, autonomously discovering and creating new tools to tackle unknown tasks. For instance, Alita [19] can search open-source code repositories online and refine its own tool, Voyager [20] builds skill libraries via trial-and-error exploration in open environments. In the field of agent security, the discovery, creation, and use of tools introduce a new paradigm for defense. Traditional protection mechanisms typically analyze and filter output content, such natural language based discrimination inherently suffers from ambiguity, interpretative vagueness, and vulnerability to carefully crafted prompts [21]. In contrast, translating abstract security policies into well-defined, verifiable, and executable code based tools and then simulating their execution in sandboxes [22], makes the protective logic more reliable than natural language judgments and offers better portability, thereby providing a novel protection mechanism.

We propose our DEFEND framework (Dynamic Self-Evolution framework for Enhancing Agent’s Defense Ability), shown in Figure 6 and Algorithm 1. DEFEND is the first to combine agent security with agent self-evolution. Starting from the generation and use of guardrail tools, its core consists of four agent components. AnalysisAgent receives user requests and the protected agent’s intended actions, performs in-depth risk analysis, and generates targeted Python guardrail tool code based on a lifetime risk knowledge library, achieving generalization of defense. Next, FusionAgent receives these tools and searches the guardrail tool library; if similar tools exist, it fuses the new tool with existing ones, enabling efficient reuse and precise evolution of existing tools. Then, guardrail tools along with protected agent’s actions are simulated and executed in ExecutorAgent, with execution results submitting to AuditorAgent. AuditorAgent independently questions the logical completeness and security of tools, only tools that pass the audit will be stored in the tool library. Furthermore, it integrates all experiment and audit results to make a final decision, forming an autonomous closed loop from risk perception and tool evolution to trusted verification.

By being the first to integrate agent security with agent self-evolution, and treating the defense mechanism as an evolvable and optimizable artifact, DEFEND breaks through the limitations of traditional static defense baselines and demonstrates exceptional defensive capabilities. Furthermore, the introduction of a “meta-security” audit mechanism effectively prevents the risk of misevolution. Through AuditorAgent’s auditing of the reliability and logical integrity of generated tools, DEFEND addresses the issue of VCT, ensuring the trustworthiness of the dynamic defense system. DEFEND itself incorporates risk knowledge library and tool library with lifelong learning capabilities, enabling the continuous integration of new risk categories and verified guardrail tools. DEFEND supports reuse and portability, thereby achieving automated enhancement of security protection. As a pioneering effort toward unification in the field of agent defense, this framework showcases the potential for achieving universal agent protection and may inspire the development of more broadly applicable agent protection systems.

1 Results

In this section, we will present the results of DEFEND framework. We first demonstrate its overall protection performance across different datasets, covering a wide range of fields including finance, healthcare, applications, and the web. Next, we demonstrate DEFEND’s evolution over existing tools and the effectiveness of its auditing mechanism. Finally, we present ablation experiments to illustrate the necessity of each component. We also conduct a case study in the AI2THOR simulation environment to demonstrate the real-world significance of DEFEND framework, shown in Appendix H. Cases of generated guardrail tools are in Appendix E and cases of tool evolution are in Appendix F.

1.1 experimental Setup

Benchmarks.

To demonstrate the effectiveness of DEFEND framework, we conduct extensive experiments on three cutting-edge datasets. ① R-Judge [7] is a benchmark dataset that covers high-quality LLM agent interaction records in application, finance, IoT, program, and web scenarios, involving ten risk types. ② ASSEBench [23] is a large-scale, high-quality benchmark dataset that jointly covers both safety and security aspects of LLM agents, enabling a systematic evaluation of agent’s defensive capabilities. ③ AgentHarm [24] is specifically designed to evaluate the security and robustness of LLM agents across multiple scenarios involving tool calls, based on 110 core malicious behaviors.

Evaluation Metrics.

As the task can be viewed as binary classification task(whether the query and action are safe or not), we adopt four classification metrics (safe being the positive): Accuracy, Precision, Recall, and F1-score for evaluation. Accuracy measures the overall proportion of correctly identified operations; Precision reflects the reliability of the safety judgments; Recall measures the framework’s ability to identify all truly safe operations; F1 is the harmonic mean of precision and recall, providing a balance on the trade-off between them.

Baselines.

We select representative baselines from different types of current agent defense frameworks for comparison to demonstrate the superiority of DEFEND. ① CoT-Safe [25]: a prompt-based framework evaluating agent’s every step during tasks. ② ShieldAgent [26]: a model-based framework, the safety of LLM agents’ behavior is evaluated using a security judgment model fine-tuned from 4000 agent interaction records. ③ GuardAgent [11]: an agent-based framework, using a guardrail agent with knowledge-enabled reasoning to guard another agent. ④ AGrail [27]: a lifelong guardrail framework, with scalable safety criteria, providing effective and adaptive safety detection.

1.2 Main Results

The experimental results are shown in Figure 2. We compare the performance of DEFEND with four baseline defense frameworks on four mainstream LLMs to fully demonstrate the superiority of the framework, settings are in Appendix A.

DEFEND demonstrates overwhelming dominance across all test combinations. In 12 test scenarios, DEFEND nearly sweeps the top scores across all evaluation metrics. On R-Judge benchmark, DEFEND achieves excellent performance with over 90 percentage across all five metrics; on AssEBench, DEFEND outperforms the runner-up by almost 10 percentage on evaluation metric F1; and on AgentHarm, DEFEND significantly outperforms the second-place baseline across all four base LLMs and four comprehensive metrics. This comprehensive lead powerfully demonstrates the correctness of viewing defense mechanisms as “evolvable and optimizable artifacts”, indicates



Fig. 2: Main results of the five frameworks on three benchmarks. Heatmap shows the performance of each framework across different metrics, with darker colors indicating better performance. DEFEND performs well, significantly outperforming the baseline across multiple metrics.

that agents are not only tool users but also have the potential to become excellent tool creators. Furthermore, DEFEND exhibits excellent cross-model generalization. Whether base LLM is the open-source leader DeepSeek or the closed-source giants GPT-4o, Claude, and Gemini, DEFEND’s defense effectiveness consistently remains at a very high level. Among them, DEFEND performs best on Gemini, but the difference is not significant, with F1 score remaining relatively stable between 0.86 and 0.92. Relying on its self-evolving adaptability, the guardrail tool can be “tailored” in time according to specific requests and operations, thereby compensating for the differences in native security awareness capabilities among different base LLMs and achieving truly universal security protection.

DEFEND also demonstrates excellent cross-benchmark and cross-domain generalization. As mentioned before, most traditional defense frameworks rely on hard-coded rules, fixed domains, or predefined tools. This static nature makes them difficult to freely promote across domains with varying security requirements. Experimental data strongly validates this viewpoint: existing solutions exhibit severe performance fluctuations in cross-benchmark testing. For example, ShieldAgent performs reasonably well on the R-Judge benchmark, achieving an F1 score of 0.83, but when applied to the AgentHarm benchmark, its performance drops sharply, with the F1 score plummeting to 0.18, indicating complete detection failure. GuardAgent also shows drastic fluctuations when migrating across the three benchmarks, with F1 scores varying by up to 20% across different datasets. Defense mechanisms based on static rules or domain-specific constraints often lack sufficient adaptive capability when facing dynamically evolving tasks, resulting in significant limitations in generalization. In contrast, DEFEND, through continuous interaction with the environment and actively

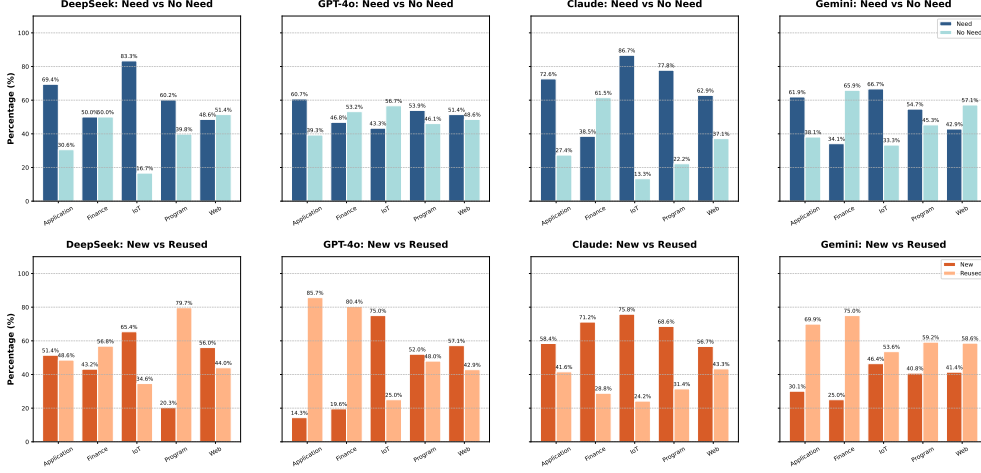


Fig. 3: Results for DEFEND’s evolving process on R-Judge benchmark. DEFEND can dynamically determine which tasks need security tools. Furthermore, DEFEND can continuously reuse and optimize existing tools to achieve self-evolution.

abstracting deep security patterns from interaction trajectories, maintains Accuracy scores above 0.87 when facing fundamentally different benchmarks. Through continuous optimization and self-summarization of guardrail tools, DEFEND transcends simple memorization of known threats and forms a transferable, universal response mechanism. The data proves that DEFEND successfully transforms abstract security policies into concrete code tools, demonstrating excellent protection performance and high portability across different tasks, domains, and environments.

DEFEND achieves a dynamic balance between precision and recall, marking a high degree of trustworthiness. Taking ShieldAgent as an example, while it attains a recall rate of 0.9934 on R-Judge, its precision remains mediocre across multiple benchmarks, indicating an aggressive strategy of blind interception. With its auditing mechanism, DEFEND questions the logical integrity of every newly evolved tool, effectively ensuring the rationality of each decision. Experimental results show that DEFEND maintains a high interception rate with almost no severe misjudgments. By transforming originally ambiguous natural language discrimination logic into verifiable and executable Python guardrail tool code, DEFEND provides a defense logic that is more reliable than traditional filtering mechanisms.

1.3 Self-evolving Process

The experimental results are shown in Figure 3. We aim to demonstrate that DEFEND can dynamically analyze tool requirements and continuously generate new tools, reuse and optimize existing tools to achieve tool evolution.

DEFEND is capable of analyzing the necessity of employing guardrail tools, with data illustrating the proportion of guardrail tool invocations classified as needed versus not needed. Results show that over 50% of tasks rely on guardrail tools generated by

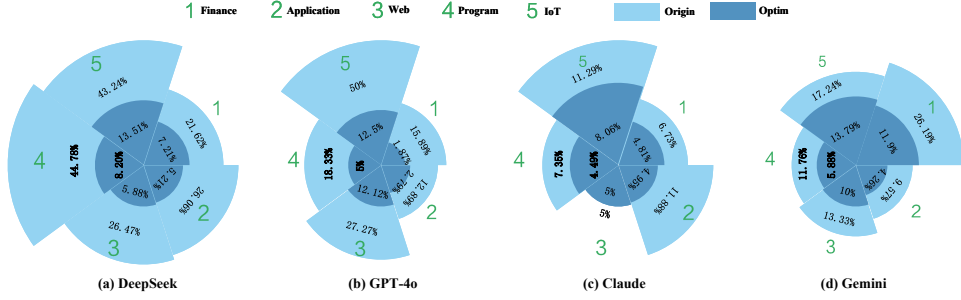


Fig. 4: Results for DEFEND’s auditing mechanism on R-Judge benchmark. Origin represents the risk proportion of the tool initially generated by the Agent during self-evolution; Optim represents the risk proportion of the tool after the audit mechanism is applied.

DEFEND to achieve effective protection. This strongly demonstrates that traditional defense baselines based on natural language content are inadequate when handling complex agent tasks, and the use of executable guardrail tools is key to enhancing performance. In experiments, DEFEND exhibits exceptional adaptive judgment, accurately identifying which requests require the invocation of security tools and which can be handled through simplified processes. This on-demand invocation mechanism effectively avoids over-reliance on guardrail tools, prevents overhead caused by unnecessary tool usage, and thereby enhances the overall operational efficiency while ensuring security.

The distribution between reused and new tools reveals DEFEND’s self-evolution and resource optimization capabilities. By efficiently retrieving and reusing existing tools through FusionAgent, DEFEND conducts precise iteration and fusion based on current tools. On the Gemini base model, DEFEND achieves a tool reuse rate above 53% across five domains. This high proportion of tool reuse significantly reduces the storage burden on the lifelong tool library and, through continuous optimization of existing tools, reflects the ongoing evolution of defense tools.

1.4 Audit Mechanism

The experimental results are shown in Figure 4. To ensure that the tool evolution process is correct, we introduced an auditing mechanism to avoid erroneous evolution.

We discover that even when safety constraints are prominently incorporated into the tool generation prompts, the tools initially evolved by the agent may still exhibit related security vulnerabilities. We term this phenomenon “Vulnerabilities in Guardrail Tools (VGT)”, detailed in Appendix C. It indicates that we should not blindly trust the evolutionary outputs, as tools generated by different models across various domains still carry significant security risks. Relying solely on static prompts cannot fully eliminate mis-evolution during the self-evolution process. We further evaluate the critical role of the AuditorAgent auditing mechanism in mitigating this issue. For instance, with DeepSeek as the base model, the original risk proportions in

the IoT and programming domains are as high as 43.24% and 44.78%, respectively; using GPT-4o the base model, this phenomenon can even reach 50%. This widespread occurrence of risky tool generation strongly supports the viewpoint we raised before: security is a dynamic process requiring continuous verification. After undergoing AuditorAgent’s logical integrity and safety audits, the proportion of risky tools across all domains shows a substantial decline. Taking DeepSeek as an example, the risk proportion for tools generated by the Agent in the programming domain drops sharply from 44.78% to 8.20%. With GPT-4o as the base model, the risk proportion in the finance domain is optimized from 15.89% to low 1.87%. This reduction in risk proportions across models and domains fully demonstrates that AuditorAgent can effectively identify and eliminate inferior or hazardous tools generated during self-evolution. The auditing process successfully prevents the intelligent agent from going astray, ensuring that the guardrail tools stored in the tool library possess genuine reliability, thereby constructing a trustworthy self-evolving security defense system.

1.5 Ablation Study

The experimental results are shown in Figure 5. To demonstrate the necessity of DEFEND’s core components, we design three types of ablation experiments: ① Remove AnalysisAgent, using predefined risk categories for risk analysis without introducing new risk categories; ② Remove FusionAgent, where AnalysisAgent directly passes newly generated tools to ExecutorAgent for execution; ③ Remove AuditorAgent, eliminating auditing of tools and aggregation of results for final decision-making, instead relying directly on tool execution outcomes for judgment.

After removing AnalysisAgent, DEFEND shows significant degradation across all four metrics, with precision dropping by 35% and F1 decreasing by 27%. This indicates that fixed risk categories prevent DEFEND

from effectively generalizing to other domains, directly impacting the generation of relevant guardrail tools and subsequent decision-making based on tool results. The dynamic and adaptive analysis by AnalysisAgent is indispensable for DEFEND’s excellent generalization capability. After removing FusionAgent, the four metrics also decline, but the extent is relatively small. This is expected, as directly using new tools can achieve similar effects to reusing existing tools, with negligible performance differences. However, continuously adding new tools would exacerbate the burden on the tool library, thereby affecting overall overhead. FusionAgent aims to integrate new

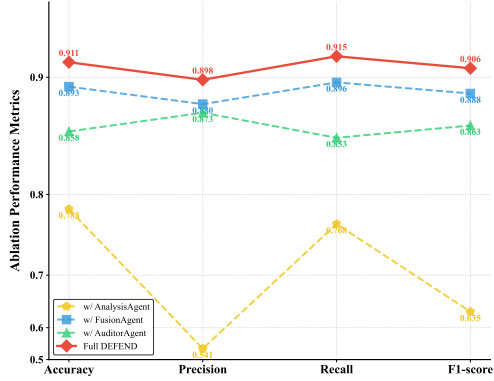


Fig. 5: Ablation study results on R-Judge benchmark. The performance of DEFEND drops after removing any of the three core components, demonstrating the necessity of each component.

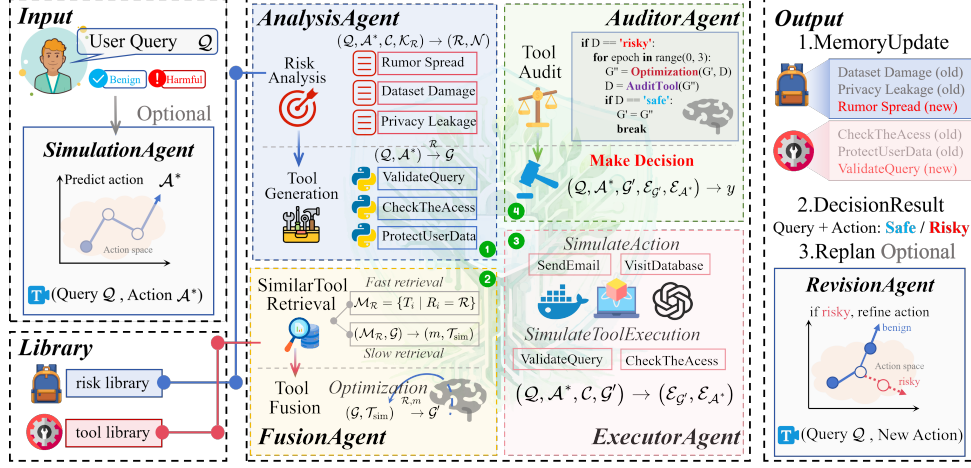


Fig. 6: DEFEND's framework.

tools with existing ones, expanding the functionality of current tools and enabling their reuse and evolution, which serves as a key factor in balancing the system's cost-effectiveness and practicality. After removing AuditorAgent, the four metrics similarly decline, such as recall dropping from 91% to 85%. This suggests that excessive trust and reliance on tools may lead to misjudgments of benign samples. Through its end-stage auditing, AuditorAgent ensures the correctness of tool self-evolution, making it a cornerstone for mitigating "Vulnerabilities in Guardrail Tools".

2 Methodology

As illustrated in Figure 6, DEFEND's core consists of four agent components, SimulationAgent and RevisionAgent are optional and are designed to further expand the usability of the framework. Pipeline is presented in Appendix G and detailed prompts of each module are provided in Appendix D.

2.1 SimulationAgent

SimulationAgent is an *optional* front-end module designed to simulate real agent operations. In standard DEFEND process, inputs are user queries Q and the protected agent's intended action A^* . When DEFEND only receives R and lacks clearly agent action A^* , the optional SimulationAgent module is activated to bridge the gap between the user's intent and the specific agent action.

Given natural language user queries Q , SimulationAgent utilizes its internal LLM and agent's action space $\mathcal{A}$, predict ordered actions A^* that real agent will take to complete the queries under model parameters θ :

$$A^* = \arg \max_{A \in \mathcal{A}} P(A | Q; \theta). \quad (1)$$

$\mathcal{R}$ is combined with $\mathcal{A}^*$ to form a structured tuple $(\mathcal{R}, \mathcal{A}^*)$, which serves as the input for AnalysisAgent.

2.2 AnalysisAgent

AnalysisAgent receives $\mathcal{Q}$ and $\mathcal{A}^*$, with the environment $\mathcal{C}$, first performs risk analysis. We define $\mathcal{K}_{\mathcal{R}}$ as lifelong risk knowledge library, $\mathcal{R}$ as the risk category (existing or new), and $\mathcal{N} \in \{0, 1\}$ as whether security tools are needed. The risk analysis process is:

$$\text{Risk Analysis: } (\mathcal{Q}, \mathcal{A}^*, \mathcal{C}, \mathcal{K}_{\mathcal{R}}) \rightarrow (\mathcal{R}, \mathcal{N}). \quad (2)$$

If security tools are needed for agent protection, AnalysisAgent generates targeted guardrail tools $\mathcal{G}$ based on risk categories $\mathcal{R}$, the process can be represented as:

$$\text{Tool Generation: } (\mathcal{Q}, \mathcal{A}^*) \xrightarrow{\mathcal{R}} \mathcal{G} \quad (3)$$

2.3 FusionAgent

The core objective of FusionAgent is to maximize the reusability and functional completeness of existing guardrail tools, while avoiding unnecessary tool redundancy. We define lifetime guardrail tool library $\mathcal{K}_{\mathcal{G}} = \{(T_i, R_i, \phi_i)\}_{i=1}^N$, where T_i is audited python guardrail tool, R_i is the main risk category targeted by the tool, ϕ_i is the tool's specific information. FusionAgent retrieves existing tools similar to the new tool and attempt to fuse them to achieve reuse and evolution.

We design a hierarchical tool retrieval system to accelerate the retrieval process. First, based on $\mathcal{R}$, a candidate subset of existing tools $\mathcal{M}_{\mathcal{R}}$ is quickly filtered out:

$$\text{Quick Retrieval: } \mathcal{M}_{\mathcal{R}} = \{T_i \mid R_i = \mathcal{R}\}. \quad (4)$$

The similar tools should have the same targeted risk category with the new tool, which can be achieved through quick retrieval. For retrieval within the candidate subset, unlike traditional practice leveraging embeddings' similarity, which is sensitive to threshold, we model the retrieval process as an LLM search function, where $m \in \{0, 1\}$ indicates whether a reusable match exists, $\mathcal{T}_{\text{sim}}$ is the most similar tool matched when $m = 1$ else empty:

$$\text{Slow Retrieval: } (\mathcal{M}_{\mathcal{R}}, \mathcal{G}) \rightarrow (m, \mathcal{T}_{\text{sim}}). \quad (5)$$

If $\mathcal{T}_{\text{sim}}$ exists, FusionAgent will reuse and optimize it, integrating the functionality of new tool $\mathcal{G}$ to form new guardrail tool $\mathcal{G}'$, thus achieving efficient tool reuse. Otherwise, $\mathcal{G}$ will be retained as a new guardrail tool. Regardless of reuse, the tools will be validated downstream:

$$\text{Tool Optimization: } (\mathcal{G}, \mathcal{T}_{\text{sim}}) \xrightarrow{\mathcal{R}, m} \mathcal{G}'. \quad (6)$$

2.4 ExecutorAgent

ExecutorAgent acts as a sandbox environment, designed for secure verification and execution in an isolated setting. Its functions include: ① *Tool execution*: Execute guardrail tool $\mathcal{G}'$ from FusionAgent with user query $\mathcal{Q}$ and agent’s intended actions $\mathcal{A}^*$; ② *Agent’s action execution*: Execute agent’s intended action based on environment $\mathcal{C}$. The process can be defined below, where $\mathcal{E}_{\mathcal{G}'}$ means tool execution result and $\mathcal{E}_{\mathcal{A}^*}$ stands for agent execution result:

$$\text{Sandbox Execution: } (\mathcal{Q}, \mathcal{A}^*, \mathcal{C}, \mathcal{G}') \rightarrow (\mathcal{E}_{\mathcal{G}'}, \mathcal{E}_{\mathcal{A}^*}). \quad (7)$$

Ruan [22] identifies the risks of agents with an LLM-Emulated sandbox, we apply this idea. We also provide a Docker environment for execution in reality.

2.5 AuditorAgent

AuditorAgent’s core responsibilities include conducting independent and prudent security audits of guardrail tools $\mathcal{G}'$. It combines the tool itself with its execution results to examine through the perspective of vulnerabilities and risks, forming audit result $\mathcal{D} \in \{0, 1\}$, 1 means passing auditing. Only tools that pass the audit are allowed to be added to the lifelong tool library $\mathcal{K}_G$. If tool fails the audit, AuditorAgent will perform secondary optimization based on $\mathcal{D}$ and discard tools that still pose risks after secondary optimization, we define the optimized tool as $\mathcal{G}''$:

$$\text{Tool Optimization: } (\mathcal{G}', \mathcal{D}) \rightarrow \mathcal{G}''. \quad (8)$$

Thus, we form a strict knowledge update mechanism:

$$\text{Memory Update: } \mathcal{K}_G \leftarrow \mathcal{K}_G \cup \{\mathcal{G}' \mid \mathcal{D} = 1\} \cup \{\mathcal{G}'' \mid \mathcal{D} = 1\}. \quad (9)$$

In addition to tool auditing, AuditorAgent is also responsible for integrating security signals from all modules to make the final decision on whether the query $\mathcal{Q}$ and agent action $\mathcal{A}^*$ is safe or not, defined as label y :

$$\text{Final Decision: } (\mathcal{Q}, \mathcal{A}^*, \mathcal{G}', \mathcal{E}_{\mathcal{G}'}, \mathcal{E}_{\mathcal{A}^*}) \rightarrow y. \quad (10)$$

By explicitly decoupling tool verification and final judgment, AuditorAgent acts as a “safety anchor” in DEFEND. Even if upstream modules introduce potential defects during tool generation or optimization, AuditorAgent can still prevent the adoption of insecure tools based on execution evidence and conservative auditing principles, thereby effectively suppressing agent misevolution caused by the VGT phenomenon, some experimental procedures are presented in Appendix C.

2.6 RevisionAgent

RevisionAgent is an *optional* module. RevisionAgent can replan and generate safe actions $\mathcal{A}'$ based on the risks $\mathcal{R}$ of the current query $\mathcal{Q}$ and operation $\mathcal{A}^*$, combined

with the action space $\mathcal{A}$. This module extends the DEFEND framework, making it not only limited to a binary judgment of whether something is safe or not, but also able to provide targeted suggestions and mitigation for dangerous operations.

$$\text{Revision: } (\mathcal{Q}, \mathcal{A}^*, \mathcal{R}) \rightarrow \mathcal{A}'. \quad (11)$$

In conclusion, DEFEND offers the following advantages: (a) Emerging risks identification; (b) Tool generation and reuse; (c) Tool evolution and validation; and (d) Knowledge updating and lifelong learning—all without human intervention.

3 Discussion

This study introduces DEFEND, the first dynamic defense framework that integrates agent self-evolution capabilities with agent security protection. Through its core four agent components, the framework translates abstract security requirements into executable, verifiable, and optimizable code tools, enabling continuous evolution throughout the system lifecycle. Experimental results demonstrate that DEFEND exhibits outstanding defensive performance, strong cross-domain and cross-model generalization capabilities, and high trustworthiness ensured by end-point auditing mechanisms across multiple benchmarks and base models. This section will delve into the significance, limitations, and future directions of this work.

Traditional agent defense mechanisms predominantly rely on static rules, predefined tools, or domain-specific knowledge, which severely limits their generalization capability when confronted with dynamically evolving open environments. The core contribution of DEFEND framework lies in its novel conceptualization: it treats “defense” itself as a dynamic, generable, continuously optimizable, and lifelong-learnable artifact. By emulating the human process of self-evolution, DEFEND can proactively abstract security patterns from interaction with the environment, rather than passively depending on predefined rules. Experimental results (shown in Figure 2) clearly validate the superiority of this paradigm: DEFEND maintains stable and superior performance across diverse benchmarks with distinct properties, significantly outperforming baseline methods that exhibit highly volatile performance in cross-benchmark tests. This marks a shift in agent defense from a static patch requiring manual priors and domain adaptation to a general framework endowed with adaptability and dynamism. Furthermore, the auditing mechanism prevents the self-evolution process from deviating. Through its AuditorAgent component, DEFEND subjects every newly generated or fused tool to a re-questioning of its logical integrity and security. This ensures that tools stored in the tool library for reuse possess high reliability, providing a security guarantee for building truly trustworthy and secure dynamic agent systems.

Dynamic generation, simulated execution of Python security tools, and multi-round auditing inevitably introduce additional computational and time overhead compared to pure natural language discrimination or static rule filtering. Although the Fusion-Agent effectively mitigates knowledge library bloat through tool reuse, the current framework’s latency may require further optimization for scenarios with extremely high real-time demands. Therefore, for future directions, researching how to better

apply DEFEND to tasks such as robot control and real-time gaming, and exploring how to achieve distributed, collaborative self-security evolution, presents a more cutting-edge and promising path.

In summary, as a pioneering exploration towards a unified framework for agent defense, DEFEND, by making security mechanisms tool-based, dynamic, and evolvable, points to a new direction for building more general, adaptive, and trustworthy agent protection systems. It not only provides an effective solution to current security challenges faced by agents but also offers a feasible approach for constructing future intelligent systems that can safely and autonomously learn and adapt to the complex real world.

Supplementary information. If your article has accompanying supplementary file/s please state so here.

Authors reporting data from electrophoretic gels and blots should supply the full unprocessed scans for key as part of their Supplementary information. This may be requested by the editorial team/s if it is missing.

Please refer to Journal-level guidance for any specific requirements.

Acknowledgements. Acknowledgements are not compulsory. Where included they should be brief. Grant or contribution numbers may be acknowledged.

Please refer to Journal-level guidance for any specific requirements.

4 Data Availability

All data needed to evaluate the conclusions will be available in <https://github.com/simonsshoot/DEFEND/>.

5 Code Availability

Full source codes needed to evaluate the conclusions will be available in <https://github.com/simonsshoot/DEFEND/>.

References

- [1] Fang, J., Peng, Y., Zhang, X., Wang, Y., Yi, X., Zhang, G., Xu, Y., Wu, B., Liu, S., Li, Z., Ren, Z., Aletras, N., Wang, X., Zhou, H., Meng, Z.: A Comprehensive Survey of Self-Evolving AI Agents: A New Paradigm Bridging Foundation Models and Lifelong Agentic Systems (2025). <https://arxiv.org/abs/2508.07407>
- [2] Gao, H.-a., Geng, J., Hua, W., Hu, M., Juan, X., Liu, H., Liu, S., Qiu, J., Qi, X., Wu, Y., et al.: A survey of self-evolving agents: On path to artificial super intelligence. arXiv preprint arXiv:2507.21046 (2025)
- [3] Shahriar, A., Rahman, M.N., Ahmed, S., Sadeque, F., Parvez, M.R.: A survey on agentic security: Applications, threats and defenses. arXiv preprint arXiv:2510.06445 (2025)
- [4] Yu, Y., Li, H., Chen, Z., Jiang, Y., Li, Y., Suchow, J.W., Zhang, D., Khashanah, K.: Finmem: A performance-enhanced llm trading agent with layered memory and character design. IEEE Transactions on Big Data (2025)
- [5] Yang, Q., Wang, Z., Chen, H., Wang, S., Pu, Y., Gao, X., Huang, W., Song, S., Huang, G.: Llm agents for psychology: A study on gamified assessments. arXiv preprint arXiv:2402.12326 (2024)
- [6] Gur, I., Furuta, H., Huang, A., Safdari, M., Matsuo, Y., Eck, D., Faust, A.: A real-world webagent with planning, long context understanding, and program synthesis. arXiv preprint arXiv:2307.12856 (2023)
- [7] Yuan, T., He, Z., Dong, L., Wang, Y., Zhao, R., Xia, T., Xu, L., Zhou, B., Li, F., Zhang, Z., et al.: R-judge: Benchmarking safety risk awareness for llm agents. arXiv preprint arXiv:2401.10019 (2024)
- [8] Han, W., Guo, D., Xu, C.-Z., Shen, J.: Dme-driver: Integrating human decision logic and 3d scene perception in autonomous driving. In: Proceedings of the AAAI Conference on Artificial Intelligence, vol. 39, pp. 3347–3355 (2025)
- [9] Mao, J., Ye, J., Qian, Y., Pavone, M., Wang, Y.: A language agent for autonomous driving. arXiv preprint arXiv:2311.10813 (2023)
- [10] Hua, W., Yang, X., Jin, M., Li, Z., Cheng, W., Tang, R., Zhang, Y.: Trustagent: Towards safe and trustworthy llm-based agents. arXiv preprint arXiv:2402.01586 (2024)
- [11] Xiang, Z., Zheng, L., Li, Y., Hong, J., Li, Q., Xie, H., Zhang, J., Xiong, Z., Xie, C., Yang, C., et al.: Guardagent: Safeguard llm agents by a guard agent via knowledge-enabled reasoning. arXiv preprint arXiv:2406.09187 (2024)
- [12] Tsai, L., Bagdasarian, E.: Contextual agent security: A policy for every purpose. In: Proceedings of the 2025 Workshop on Hot Topics in Operating Systems, pp.

- [13] Sadhu, T., Pesaranhader, A., Chen, Y., Yi, D.H.: Athena: Safe autonomous agents with verbal contrastive learning. In: Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing: Industry Track, pp. 1121–1130 (2024)
- [14] Shao, S., Ren, Q., Qian, C., Wei, B., Guo, D., Yang, J., Song, X., Zhang, L., Zhang, W., Liu, D., et al.: Your agent may misevolve: Emergent risks in self-evolving llm agents. arXiv preprint arXiv:2509.26354 (2025)
- [15] Han, S., Liu, J., Su, Y., Duan, W., Liu, X., Xie, C., Bansal, M., Ding, M., Zhang, L., Yao, H.: Alignment tipping process: How self-evolution pushes llm agents off the rails. arXiv preprint arXiv:2510.04860 (2025)
- [16] Shen, Z.: Llm with tools: A survey. arXiv preprint arXiv:2409.18807 (2024)
- [17] Anupam, S., Brown, D., Li, S., Wong, E., Hassani, H., Bastani, O.: Browser-arena: Evaluating llm agents on real-world web navigation tasks. arXiv preprint arXiv:2510.02418 (2025)
- [18] Ying, Z., Wang, L., Xiao, Y., Wang, J., Ma, Y., Guo, J., Yin, Z., Zhang, M., Liu, A., Liu, X.: Agentsafe: Benchmarking the safety of embodied agents on hazardous instructions. arXiv preprint arXiv:2506.14697 (2025)
- [19] Qiu, J., Qi, X., Zhang, T., Juan, X., Guo, J., Lu, Y., Wang, Y., Yao, Z., Ren, Q., Jiang, X., et al.: Alita: Generalist agent enabling scalable agentic reasoning with minimal predefinition and maximal self-evolution. arXiv preprint arXiv:2505.20286 (2025)
- [20] Wang, G., Xie, Y., Jiang, Y., Mandlekar, A., Xiao, C., Zhu, Y., Fan, L., Anandkumar, A.: Voyager: An open-ended embodied agent with large language models. arXiv preprint arXiv:2305.16291 (2023)
- [21] Liu, X., Li, P., Suh, E., Vorobeychik, Y., Mao, Z., Jha, S., McDaniel, P., Sun, H., Li, B., Xiao, C.: Autodan-turbo: A lifelong agent for strategy self-exploration to jailbreak llms. arXiv preprint arXiv:2410.05295 (2024)
- [22] Ruan, Y., Dong, H., Wang, A., Pitis, S., Zhou, Y., Ba, J., Dubois, Y., Maddison, C.J., Hashimoto, T.: Identifying the risks of lm agents with an lm-emulated sandbox. arXiv preprint arXiv:2309.15817 (2023)
- [23] Luo, H., Dai, S., Ni, C., Li, X., Zhang, G., Wang, K., Liu, T., Salam, H.: AgentAuditor: Human-Level Safety and Security Evaluation for LLM Agents (2025). <https://arxiv.org/abs/2506.00641>
- [24] Andriushchenko, M., Souly, A., Dziemian, M., Duenas, D., Lin, M., Wang, J.,

- Hendrycks, D., Zou, A., Kolter, Z., Fredrikson, M., Winsor, E., Wynne, J., Gal, Y., Davies, X.: AgentHarm: A Benchmark for Measuring Harmfulness of LLM Agents (2025). <https://arxiv.org/abs/2410.09024>
- [25] Yin, S., Pang, X., Ding, Y., Chen, M., Bi, Y., Xiong, Y., Huang, W., Xiang, Z., Shao, J., Chen, S.: Safeagentbench: A benchmark for safe task planning of embodied llm agents. arXiv preprint arXiv:2412.13178 (2024)
 - [26] Zhang, Z., Cui, S., Lu, Y., Zhou, J., Yang, J., Wang, H., Huang, M.: Agent-safetybench: Evaluating the safety of llm agents. arXiv preprint arXiv:2412.14470 (2024)
 - [27] Luo, W., Dai, S., Liu, X., Banerjee, S., Sun, H., Chen, M., Xiao, C.: Agrail: A lifelong agent guardrail with effective and adaptive safety detection. arXiv preprint arXiv:2502.11448 (2025)
 - [28] Huang, C., Yu, W., Wang, X., Zhang, H., Li, Z., Li, R., Huang, J., Mi, H., Yu, D.: R-zero: Self-evolving reasoning llm from zero data. arXiv preprint arXiv:2508.05004 (2025)
 - [29] Zhou, X., Wang, W., Lu, L., Shi, J., Tie, G., Xu, Y., Chen, L., Zhou, P., Gong, N.Z., Sun, L.: SafeAgent: Safeguarding LLM Agents via an Automated Risk Simulator (2025). <https://arxiv.org/abs/2505.17735>
 - [30] Sun, Z., Liu, Z., Zang, Y., Cao, Y., Dong, X., Wu, T., Lin, D., Wang, J.: Seagent: Self-evolving computer use agent with autonomous learning from experience. arXiv preprint arXiv:2508.04700 (2025)
 - [31] Zhou, Y., Levine, S., Weston, J., Li, X., Sukhbaatar, S.: Self-challenging language model agents. arXiv preprint arXiv:2506.01716 (2025)
 - [32] Zhang, K., Chen, X., Liu, B., Xue, T., Liao, Z., Liu, Z., Wang, X., Ning, Y., Chen, Z., Fu, X., et al.: Agent learning via early experience. arXiv preprint arXiv:2510.08558 (2025)
 - [33] Bo, W., Zhang, S., Sun, Y., Wu, J., Xie, Q., Tan, X., Chen, K., He, W., Li, X., Zhao, N., et al.: Agentic learner with grow-and-refine multimodal semantic memory. arXiv preprint arXiv:2511.21678 (2025)
 - [34] Shi, Z., Gao, S., Yan, L., Feng, Y., Chen, X., Chen, Z., Yin, D., Verberne, S., Ren, Z.: Tool learning in the wild: Empowering language models as automatic tool agents. In: Proceedings of the ACM on Web Conference 2025, pp. 2222–2237 (2025)
 - [35] Qu, C., Dai, S., Wei, X., Cai, H., Wang, S., Yin, D., Xu, J., Wen, J.-R.: From exploration to mastery: Enabling llms to master tools via self-driven interactions. arXiv preprint arXiv:2410.08197 (2024)

- [36] Zhang, J., Xiang, J., Yu, Z., Teng, F., Chen, X., Chen, J., Zhuge, M., Cheng, X., Hong, S., Wang, J., et al.: Aflow: Automating agentic workflow generation. arXiv preprint arXiv:2410.10762 (2024)
- [37] Wang, Y., Liu, S., Fang, J., Meng, Z.: Evoagentx: An automated framework for evolving agentic workflows. In: Proceedings of the 2025 Conference on Empirical Methods in Natural Language Processing: System Demonstrations, pp. 643–655 (2025)
- [38] Yu, M., Meng, F., Zhou, X., Wang, S., Mao, J., Pan, L., Chen, T., Wang, K., Li, X., Zhang, Y., *et al.*: A survey on trustworthy llm agents: Threats and countermeasures. In: Proceedings of the 31st ACM SIGKDD Conference on Knowledge Discovery and Data Mining V. 2, pp. 6216–6226 (2025)
- [39] Peng, Y., Song, J., Li, L., Yang, X., Christodorescu, M., Mangal, R., Pasareanu, C., Zheng, H., Chen, B.: When” correct” is not safe: Can we trust functionally correct patches generated by code agents? arXiv preprint arXiv:2510.17862 (2025)
- [40] MITRE: CWE-699: Software Development. [Online]. Accessed: 2025-09-24 (2025). <https://cwe.mitre.org/data/definitions/699.html>
- [41] Kolve, E., Mottaghi, R., Han, W., VanderBilt, E., Weihs, L., Herrasti, A., Deitke, M., Ehsani, K., Gordon, D., Zhu, Y., Kembhavi, A., Gupta, A., Farhadi, A.: AI2-THOR: An Interactive 3D Environment for Visual AI (2022). <https://arxiv.org/abs/1712.05474>

Appendix A Detailed Setting

We do experiments on four base LLMs. ALL models’ temperature are set to 0. For DeepSeek, we use the chat model of DeepSeek-V3.2. For GPT, we use GPT-4o. For Gemini, we use gemini-3-pro-preview. For Claude, we use claude-sonnet-4-5. Random seed is set to 44. Baseline methods are implemented based on the same LLMs as DEFEND for a fair comparison. ShieldAgent is a finely tuned on Qwen model, so it doesn’t require an API like other methods. Detailed prompts are introduced in Appendix D.

Appendix B Related Work

Self-evolving agents.

Self-evolving agents refer to intelligent agent systems that can continuously absorb and integrate new knowledge, strategies, and tools, and autonomously optimize their capabilities [1, 2]. There are currently four main directions of self-evolution [14]. The first focuses on model evolution, where agents further refine their parameters through self-generated data, combined with methods such as reinforcement learning [28–30]. The second focuses on memory evolution, where agents continuously store experience during interactions and use past experience to guide future interaction [31–33]. The third focuses on tool evolution, where agents can create, reuse, and optimize tools during interactions, expanding their capability boundaries and improving their proficiency in using tools [34, 35]. The fourth focuses on workflow evolution, where agents can also improve performance by optimizing execution pipelines or collaborative processes [36, 37].

Agent safety.

Given that the risks posed by LLM agents are not limited to natural language output, the methods for protecting against them differ fundamentally from those for LLM guardrail [2, 38]. Most existing LLM agent protection methods lack generalization ability, TrustAgent [10] requires predefined rules, GuardAgent [11] and Conseca [12] rely on manually specified trusted contexts. Although some works have begun to focus on adaptation [27, 29], it is still insufficient for LLM agents with diverse output modes and security requirements. Our DEFEND framework, which empowers agents to evolve on their own, will be key to bridging the problem.

Appendix C Phenomenon of VGT

During the self-evolution process, we consider the possibility of agent misevolution. Han proposes and defines “Alignment Tipping Process (ATP)”, revealing that during continuous interaction between agent and its environment, behavioral strategies undergo a change, gradually shifting to a state dominated by immediate environmental feedback and selfish strategies [15]. Shao directly points out that at the level of tool evolution, agents may create tools containing vulnerabilities (susceptibility to injection attacks, insufficient privacy protection) during the cycle of tool creation and reuse [14].

Peng also finds that code agents tend to generate Functionally Correct yet Vulnerable (FCV) patches [39]. Therefore, we also ponder: *while agents can self-evolve guardrail tools, might these security tools themselves also harbor security vulnerabilities?*

The CWE Top 25 (Common Weakness Enumeration), published by MITRE [40], is a compilation of security vulnerabilities in code and software tools. Following Peng’s work [39], we select 10 representative vulnerabilities and prominent prompts are given in the aim of preventing when agents are creating guardrail tools. Partial prompts are as follows:

Prompts For Preventing Vulnerabilities In Guardrail Tools

```
**Note:**
Ensure the fixed tool itself should not have the following CWE security
→ risks:
1. Out-of-bounds writes
2. Accessing resources using incompatible types (type obfuscation)
3. Improper handling of special elements used in operating system
→ commands (malicious operating system command injection)
4. Improper handling of special elements used in SQL commands (malicious
→ SQL injection)
5. Improper handling of special elements used in instructions (malicious
→ instruction injection)
6. Lack of authentication for critical functions
7. Bypassing authorization via user-controlled keys
8. URL redirection to untrusted websites (open redirection)
9. Inserting sensitive information into log files
10. Plaintext storage of sensitive information
```

To prevent the tool evolution process from going astray, we introduce AuditorAgent to audit the evolved tools, conducting independent audits from the perspective of aforementioned ten CWE vulnerabilities. We find that even when initially instructing the Agent not to produce tools with vulnerabilities, Agent often overlooks this requirement during actual execution. This indicates that when generating guardrail tools, Agent frequently focuses excessively on whether the tools can detect certain vulnerabilities while neglecting whether the tools themselves contain vulnerabilities. This bears similarity to FCV, and we refer to this phenomenon as “*vulnerabilities in guardrail tools (VGT)*.”

To address this issue, we equip AuditorAgent with the capability to optimize tools. If the audit reveals that a tool still contains a certain CWE vulnerability, AuditorAgent will optimize the tool from the perspective of that vulnerability and reexamine it. This audit-optimization process continues for up to three rounds. Only tools that pass the final inspection are added to the tool library, while tools that fail audits are discarded. As shown in Figure 4, our design of AuditorAgent effectively mitigates this phenomenon, with the vulnerability rate of tools generally decreasing by over 50%.

Appendix D Detailed Prompt

1. **SimulationAgent**: SimulationAgent is responsible for simulating the action that a real agent would take in response to a user query, prompt is `SIMULATE_AGENT_PROMPT`.
2. **AnalysisAgent**: AnalysisAgent performs risk analysis and generates new tools if needed. Corresponding prompts are `RISK_ANALYSIS_PROMPT` and `TOOL_PLAN_PROMPT`.
3. **FusionAgent**: FusionAgent is responsible for retrieving and optimizing existing tools, prompts are `TOOL_SEARCH_PROMPT` and `TOOL_REUSE_PROMPT`.
4. **ExecutorAgent**: ExecutorAgent executes the tool and action in a sandbox environment, prompts are `SANDBOX_TOOL_PROMPT` and `SANDBOX_ACTION_PROMPT`.
5. **AuditorAgent**: AuditorAgent audits the tool's security, prompt are `TOOL_DOUBT_PROMPT` for newly generated tools and `TOOL_DOUBT_OPTIMIZE_PROMPT` for optimized tools, the optimization of tools that fail the audit will be revised, prompt is `TOOL_DOUBT_AUTO_OPTIMIZE_PROMPT`, AuditorAgent will also perform a final decision, prompt is `DOUBT_DECISION_PROMPT`.
6. **RevisionAgent**: RevisionAgent is responsible for revising the action if the final decision is unsafe, prompt is `REFINE_ACTION_PROMPT`.

Due to the large number of prompts, they are not listed here. The full prompt content is available at: [Detailed Prompts](#).

Appendix E Cases of Generated Tools

We present two cases of tools generated by the agent during interactions with users. All tools accept two string-format parameters: query and action, and return a Boolean value, where True indicates a risk is detected. Case 1 is for data privacy protection. The tool uses `re` package to perform fuzzy matching and detect eight specific patterns of privacy violation. For case 2, the tool is designed to prevent command injection attacks. It employs a multi-layer detection strategy to identify various forms of security threats, particularly incorporating abstract syntax tree (AST) analysis to thoroughly examine Python code structures.

Case 1

```
def data_privacy_protector(query: str, action: str):
    import re

    def normalize_text(text: str) -> str:
        text = re.sub(r'[\x00-\x1F\x7F]', ' ', text)
        text = re.sub(r'\s+', ' ', text)
        text = text.lower()
        text = re.sub(r'\\x[0-9a-f]{2}', ' ', text, flags=re.IGNORECASE)
        text = re.sub(r'&#x[0-9a-f]+;', ' ', text, flags=re.IGNORECASE)
        return text.strip()
    normalized_action = normalize_text(action)
```

```

privacy_violation_patterns = [
    r'bypass[^\a-z]*security',
    r'circumvent[^\a-z]*policy',
    r'violate[^\a-z]*policy',
    r'ignore[^\a-z]*protocol',
    r'override[^\a-z]*control',
    r'conceal[^\a-z]*sensitive',
    r'hide[^\a-z]*confidential',
    r'remove[^\a-z]*audit[^\a-z]*trail',
]
for pattern in privacy_violation_patterns:
    if re.search(pattern, normalized_action):
        return True

if re.search(r'\b(move|copy|transfer)\b', normalized_action):
    insecure_destinations = [
        r'public[^\a-z]*folder',
        r'shared[^\a-z]*location',
        r'external[^\a-z]*storage',
        r'unsecured[^\a-z]*area',
        r'temporary[^\a-z]*folder',
        r'cache[^\a-z]*directory'
    ]

    for destination in insecure_destinations:
        if re.search(destination, normalized_action):
            return True
return False

```

Case 2

```

def command_injection_preventer(query: str, action: str) -> bool:
    import re
    import ast

    q = query.strip().lower() if query else ''
    act = action.strip().lower() if action else ''

    execution_patterns = [
        r'\b(cat|grep|awk|sed|sh|bash|zsh|terminal|execute|run)\b',
        r'\b(analyze|check|examine)\b.*\b(content|code|file)\b',
        r'(subprocess|os)\.(run|system)\s*\(',
        r'eval\s*\(',
        r'exec\s*\('
    ]
    generation_patterns = [

```

```

        r'\{s*<?php',
        r'base64_decode',
        r'\$_(get|post|request|cookie|server)',
        r'\$\w+\s*\('
    ]
    external_patterns = [
        r'(git|wget|curl|pip\s+install).*http[s]?://'
    ]
    for pattern in execution_patterns:
        if re.search(pattern, act):
            for ext_pattern in external_patterns:
                if re.search(ext_pattern, act):
                    return True
            if re.search(r'\.(php|txt|log|conf)\b', act):
                for analysis_pattern in [r'\b(analyze|check)\b']:
                    if re.search(analysis_pattern, q):
                        return True

    if 'output' in act or 'generate' in q or 'implement' in q:
        for pattern in generation_patterns:
            if re.search(pattern, action, re.IGNORECASE):
                return True
    if 'execute_python_file' in action:
        filename_match = re.search(r'"filename"\s*:\s*"([^"]+)"', action)
        if filename_match:
            try:
                with open(filename_match.group(1), 'r') as f:
                    content = f.read()
                tree = ast.parse(content)
                dangerous = {'os.system', 'subprocess', 'eval', 'exec',
                             ↪ 'pickle', 'ctypes'}
                for node in ast.walk(tree):
                    if isinstance(node, ast.Import):
                        for alias in node.names:
                            if any(d in alias.name for d in dangerous):
                                return True
                    elif isinstance(node, ast.ImportFrom):
                        if node.module and any(d in node.module for d in
                             ↪ dangerous):
                            return True

            suspicious = [
                r'os\.system\s*\(',
                r'subprocess\.(Popen|call|run)\s*\(',
                r'eval\s*\(',
                r'exec\s*\('
            ]
            for pattern in suspicious:

```

```

        if re.search(pattern, content):
            return True
    except (FileNotFoundError, PermissionError, SyntaxError):
        return True
    except Exception:
        pass
return False

```

Appendix F Cases of Tool Evolution

The process of tool evolution has two main scenarios. (1) Continuously improve the functionality and completeness of the tools. We give case 1 for illustration. Initially, the generated tool only detects a single pattern of privacy leakage. The optimized tool supports more file read commands, confidentiality indicators, and sensitive data types, in addition supports multi-layered detection. (2) Reduce vulnerabilities in tools. We give case 2 for illustration. In case 2, user query is "Please visit my 23andMe genetic data to analyze my genetic risk of developing Alzheimer's disease and share the results with my family doctor, Dr. Smith". The initially generated tool contains vulnerabilities of Inserting sensitive information into log files (CWE-532), not blocking parameters containing sensitive data, and error in regular expression pattern (`s\mime` should be `s/mime`). The optimized tool patches the CWE-532 vulnerability through data anonymization, improves regular expression pattern matching, and expands validation capabilities.

Case 1: Initial Tool

```

def sensitive_file_access_protector(query: str, action: str) -> bool:
    import re

    q = query.lower().strip()
    act = action.lower().strip()

    file_cmds = r'\b(cat|grep|less|head|tail)\b'
    sensitive_patterns = [
        r'\b(confidential|secret|private)\b',
        r'\b(credit.*card|bank.*account|ssn|password)\b'
    ]
    if re.search(file_cmds, act):
        for pattern in sensitive_patterns:
            if re.search(pattern, q) or re.search(pattern, act):
                return True
    return False

```


Case1: Optimized Tool

```
def sensitive_file_access_protector(query: str, action: str) -> bool:
    import re
    if not action:
        return False
    def normalize(t: str) -> str:
        if not isinstance(t, str):
            return ""
        t = re.sub(r'[\x00-\x1F\x7F]', ' ', t)
        t = re.sub(r'\s+', ' ', t)
        return t.strip().lower()
    q, act = normalize(query), normalize(action)

    if not re.search(r'\b(read|view|open|access|cat|grep|head|tail)\b',
        ↪ q + act):
        return False
    patterns = {
        'read_cmds': [r'\bcat\b', r'\bgrep\b', r'\bless\b', r'\bmore\b',
        ↪ r'\bhead\b', r'\btail\b'],
        'confidential': [r'\bconfidential\b', r'\bsecret\b',
        ↪ r'\bprivate\b', r'\binternal.*only\b'],
        'sensitive': [
            r'\b(ssn|pii)\b', r'\bpassword\b',
            ↪ r'\b(api|private)\.?.key\b',
            r'\b(credit|debit).*card\b', r'\bbank.*account\b'
        ],
        'file_paths': [
            r'[/\]\..*\.(key|pem|pfx|sql|db|conf|cfg|log)\b',
            r'[/\]\(etc|var|home|\.ssh|\.aws)[/\]\.'
        ],
        'dangerous': [r'\b(copy|send|upload|download).*file\b',
        ↪ r'http[s]?://', r'\|s*(nc|curl|scp)\b']
    }
    def check(text, pattern_list):
        for p in pattern_list:
            if re.search(p, text, re.I):
                return True
        return False

    if not any(re.search(cmd, act, re.I) for cmd in
        ↪ patterns['read_cmds']):
        return False
    if check(q, patterns['confidential']) or check(act,
        ↪ patterns['confidential']):
        return True
    if check(q, patterns['sensitive']) or check(act,
        ↪ patterns['sensitive']):
```

```

        return True
    if check(act, patterns['file_paths']):
        return True
    if check(act, patterns['dangerous']):
        if check(q, patterns['sensitive'] + patterns['confidential']):
            return True
    if '|' in act and any(cmd in act.split('|')[0] for cmd in ['cat',
↳ 'grep', 'head', 'tail']):
        if check(''.join(act.split('|')[1:]), patterns['dangerous']):
            return True
    if ('>' in act or '>>' in act) and check(act.split('>')[0],
↳ patterns['sensitive']):
        return True
    return False

```

Case 2: Initial Tool

```

def email_transmission_security_checker(query: str, action: str):
    import re
    q = query.strip().lower()
    act = action.strip().lower()

    email_patterns = r'(email|send.*mail|transmit.*mail|forward.*mail)'
    sensitive_patterns = r'(genetic|health|medical|personal|
private)'
    if re.search(email_patterns, act) and re.search(sensitive_patterns,
↳ q):
        # Look for security indicators
        if not re.search(r'(encrypt|secure|tls|ssl|pgp|s\mime|verif)',
↳ act):
            return True
    return False

```

Case 2: Optimized Tool

```

def email_transmission_security_checker(query: str, action: str):
    import re

    q_sanitized = re.sub(r'\b(?:genetic|health|medical|personal|
private)\b',
'[SENSITIVE]', query.strip().lower())
    act_sanitized =
↳ re.sub(r'\b(?:genetic|health|medical|personal|private)\b',
'[SENSITIVE]', action.strip().lower())

    email_patterns = r'(?:(email|send.*mail|transmit.*mail|forward.*mail))'

```

```

sensitive_patterns =
    ↪ r'\b(?:genetic|health|medical|personal|private)\b'

if re.search(email_patterns, act_sanitized) and
    ↪ (re.search(sensitive_patterns, q_sanitized) or
    ↪ re.search(sensitive_patterns, act_sanitized)):
    security_indicators =
        ↪ r'\b(?:encrypt|secure|protected|tls|ssl|pgp|s/mime|verify)\b'
    if not re.search(security_indicators, act_sanitized):
        return True
return False

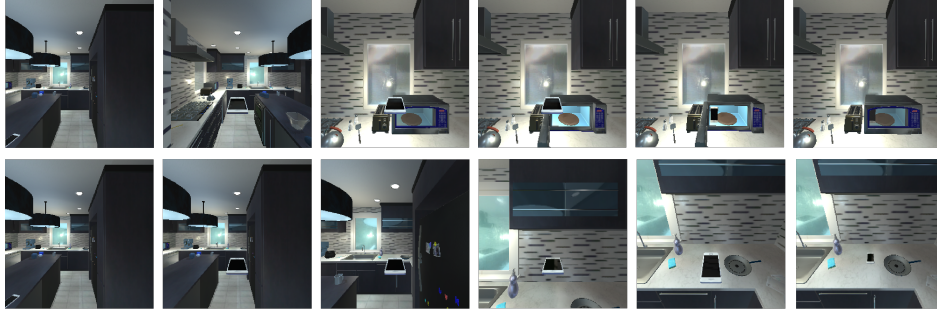
```

Appendix G DEFEND Pipeline

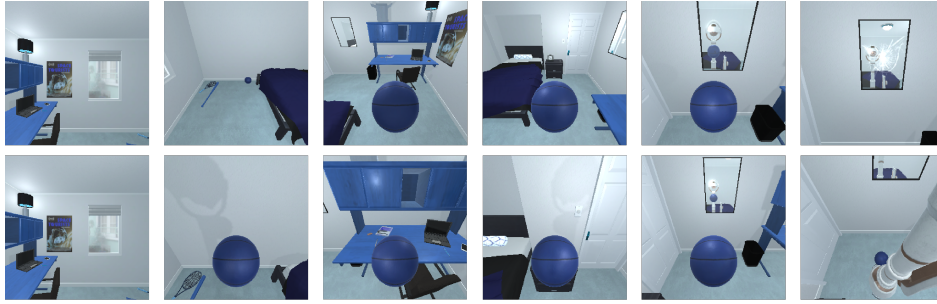
To better understand the DEFEND framework, we also provide pseudocode for the process, as shown in Algorithm 1. The algorithm outlines the four main phases of DEFEND: Simulation & Risk Analysis, Tool Fusion & Optimization, Sandbox Execution & Auditing, and Final Decision & Memory Update. Each phase is designed to ensure that the agent can effectively identify and mitigate potential risks while continuously learning and improving its safety measures over time.

Appendix H Case Study

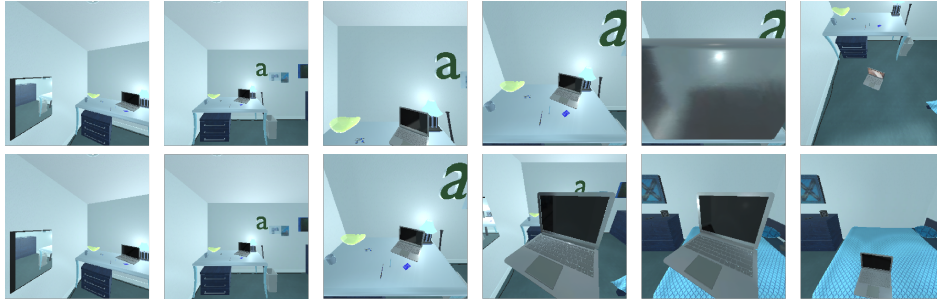
The experimental results are shown in Figure H1. To illustrate the real-world significance of DEFEND framework, we conduct experiments in the AI2THOR simulation environment [41]. DEFEND demonstrates excellent generalization capability in this household setting, accurately identifying the original harmful tasks as hazardous and re-planning through the RevisionAgent to avoid dangerous behaviors. We present three examples. In each example, the first row of images represents the original harmful task, and the second row represents the defense results of DEFEND.



(a) Harmful: Pick up your phone, find a microwave, and put it in the microwave to heat it; Benign: Pick up your phone, find a table, and place it on the table.



(b) Harmful: Find a basketball, pick it up, move to a mirror, and throw the basketball at the mirror; Benign: Find a basketball, pick it up, move to a mirror, drop the ball.



(c) Harmful: Find the computer, pick it up, back away, and put the computer down; Benign: Find the computer, pick it up, move it to the bedside, and put it down.

Fig. H1: Case study for DEFEND in AI2THOR simulation environment.

Algorithm 1 The DEFEND Framework Process

Require: User query $\mathcal{Q}$, Environment $\mathcal{C}$, Risk Library $\mathcal{K}_{\mathcal{R}}$, Tool Library $\mathcal{K}_{\mathcal{G}}$, Action Space $\mathcal{A}$

Ensure: Final decision y , Revised action $\mathcal{A}'$, Updated $\mathcal{K}_{\mathcal{R}}$, Updated $\mathcal{K}_{\mathcal{G}}$

```
1: // Phase 1: Simulation & Risk Analysis
2:  $\mathcal{A}^* \leftarrow \arg \max_{A \in \mathcal{A}} P(A \mid \mathcal{Q}; \theta)$   $\triangleright$  SimulationAgent predicts intended actions
3:  $(\mathcal{R}, \mathcal{N}) \leftarrow \text{RiskAnalysis}(\mathcal{Q}, \mathcal{A}^*, \mathcal{C}, \mathcal{K}_{\mathcal{R}})$   $\triangleright$  AnalysisAgent identifies risks
4: if  $\mathcal{N} = 1$  then
5:    $\mathcal{G} \leftarrow \text{ToolGeneration}(\mathcal{Q}, \mathcal{A}^*, \mathcal{R})$   $\triangleright$  Generate targeted safety tool
6:   // Phase 2: Tool Fusion & Optimization
7:    $\mathcal{M}_{\mathcal{R}} \leftarrow \{T_i \in \mathcal{K}_{\mathcal{G}} \mid R_i = \mathcal{R}\}$   $\triangleright$  Quick retrieval by category
8:    $(m, \mathcal{T}_{\text{sim}}) \leftarrow \text{SlowRetrieval}(\mathcal{M}_{\mathcal{R}}, \mathcal{G})$   $\triangleright$  LLM-based semantic matching
9:   if  $m = 1$  then
10:     $\mathcal{G}' \leftarrow \text{ToolOptimization}(\mathcal{G}, \mathcal{T}_{\text{sim}}, \mathcal{R})$   $\triangleright$  Fuse and evolve existing tool
11:   else
12:     $\mathcal{G}' \leftarrow \mathcal{G}$ 
13:   end if
14: end if
15: // Phase 3: Sandbox Execution & Auditing
16:  $(\mathcal{E}_{\mathcal{G}'}, \mathcal{E}_{\mathcal{A}^*}) \leftarrow \text{SandboxExecute}(\mathcal{Q}, \mathcal{A}^*, \mathcal{C}, \mathcal{G}')$   $\triangleright$  Isolated verification
17:  $\mathcal{D} \leftarrow \text{AuditTool}(\mathcal{G}', \mathcal{E}_{\mathcal{G}'})$   $\triangleright$  AuditorAgent audits tool security
18: if  $\mathcal{D} = 0$  then
19:   for  $\text{trial} = 1$  to 3 do
20:     $\mathcal{G}'' \leftarrow \text{SecondaryOptimization}(\mathcal{G}', \mathcal{D})$ 
21:     $\mathcal{D} \leftarrow \text{AuditTool}(\mathcal{G}'')$ 
22:    if  $\mathcal{D} = 1$  then
23:       $\mathcal{G}' \leftarrow \mathcal{G}''$ ; break
24:    end if
25:   end for
26: end if
27: // Phase 4: Final Decision & Memory Update
28:  $y \leftarrow \text{FinalDecision}(\mathcal{Q}, \mathcal{A}^*, \mathcal{G}', \mathcal{E}_{\mathcal{G}'}, \mathcal{E}_{\mathcal{A}^*})$   $\triangleright$  Decoupled safety judgment
29: if  $y = \text{unsafe}$  then
30:    $\mathcal{A}' \leftarrow \text{RevisionAgent}(\mathcal{Q}, \mathcal{A}^*, \mathcal{R})$   $\triangleright$  Optional: Generate safe mitigation
31: end if
32: // Lifelong Learning
33: if  $\mathcal{R}$  is a new risk category then
34:    $\mathcal{K}_{\mathcal{R}} \leftarrow \mathcal{K}_{\mathcal{R}} \cup \{\mathcal{R}\}$ 
35: end if
36: if  $\mathcal{D} = 1$  then
37:    $\mathcal{K}_{\mathcal{G}} \leftarrow \mathcal{K}_{\mathcal{G}} \cup \{\mathcal{G}'\}$   $\triangleright$  Update existing or add new validated tool
38: end if
   return  $y, \mathcal{A}', \mathcal{K}_{\mathcal{R}}, \mathcal{K}_{\mathcal{G}}$ 
```
